WEB TASARIMI VE PROGRAMLAMA

LAB-5

JavaScript/TypeScript Temelleri · DOM Yerine "State" Mantığı

İçindekiler

1 LAB-5 Öğrenme Hedefleri **2**

2 Modern JavaScript (ES2020+) Temelleri **2**

3 ES Modules: import / export **5**

4 Async/Await ve Fetch API **6**

5 Error Handling (Hata Yönetimi) **9**

6 TypeScript'e Giriş: Neden Tip Güvenliği? **11**

7 TypeScript Temel Tipler **12**

8 Interface ve Type Alias **14**

9 Generics (Temel Düzey) **17**

10 DOM Yerine "State" Mantığı **19**

11 Uygulama: JSON'dan Proje Verisi Çekme, Filtreleme ve Sıralama **21**

12 Git İş Akışı ve Teslim **27**

13 Teslim Kontrol Listesi ve Değerlendirme Rubriği **28**

13.1 Değerlendirme Rubriği 28

14 Kendini Test Et **29**

15 Ek Kaynaklar ve İleri Okuma **30**

1 LAB-5 Öğrenme Hedefleri

Önceki laboratuvarlarda geliştirme ortamını kurmuş (LAB-1), semantik ve erişilebilir bir HTML yapısı oluşturmuş (LAB-2), bunu responsive CSS ile düzenlemiş (LAB-3) ve Tailwind CSS ile component tabanlı bir UI kütüphanesi kurmuştun (LAB-4). Artık sayfalar görsel olarak hazır; şimdi sıra bu sayfalara **mantık ve veri** katmaya geldi.

Bu laboratuvarda modern JavaScript (ES2020+) özelliklerini ve TypeScript'in temellerini öğreneceksin. Geleneksel "DOM manipülasyonu" yaklaşımı yerine, React'in temelini oluşturan **state (durum) mantığını** kavrayacaksın. JSON verisinden proje listesi çekip filtreleme ve sıralama yapan, TypeScript tipleri ile güvence altına alınmış çalışan bir sayfa üreteceksin.

Bu laboratuvarın sonunda aşağıdaki becerilere sahip olacaksın:

1. **ES Modules kullanmak:** `import/export` ile kodunu modüller halinde organize etmek.
2. **Async/await ve Fetch API kullanmak:** Asenkron veri çekme işlemlerini modern sözdizimi ile yazmak.
3. **Hata yönetimi (error handling) uygulamak:** `try/catch/finally` ile hataları zarif şekilde yakalamak.
4. **TypeScript temel tiplerini kullanmak:** `string`, `number`, `boolean`, `array`, `union`, `literal` tiplerini doğru uygulamak.
5. **Interface ve type alias tanımlamak:** Veri yapılarını TypeScript ile modellemek.
6. **Generics'i temel düzeyde anlamak:** Yeniden kullanılabilir, tip-güvenli fonksiyonlar yazmak.
7. **Veri modeli oluşturmak:** `Project` tipinde bir veri modeli tanımlamak ve JSON verisinden çekilen projeleri listelemek, filtrelemek ve sıralamak.

Benzetme ile Anlama

LAB-1–4'te binanın dış cephesini — duvarları, boyayı, pencere çerçevelerini — tamamladık. Şimdi binanın **elektrik ve su tesisatını** döşüyoruz: Dışarıdan görünmez ama bina bu altyapı olmadan "çalışmaz". JavaScript/TypeScript, web uygulamasının görünmez ama vazgeçilmez altyapısıdır; veriyi taşır, işler ve kullanıcıya sunar.

Sırada Ne Var?

Hedeflerimiz belirlendi. İlk adım olarak modern JavaScript'in (ES2020+) temel özelliklerini hızlıca gözden geçirelim.

2 Modern JavaScript (ES2020+) Temelleri

JavaScript sürekli gelişen bir dildir. ECMAScript (ES) standartları her yıl güncellenir. Bu bölümde LAB boyunca kullanacağımız temel modern JavaScript özelliklerini öğreneceksin.

Teorik Arka Plan

Değişken tanımlama: `const` ve `let`

```
// const: degeri degismeyen (sabit) degiskenler
const API_URL = "https://api.example.com/projects";
const MAX_ITEMS = 10;

// let: degeri degisebilen degiskenler
let currentPage = 1;
let searchQuery = "";

// var: KULLANMA! Kapsam (scope) sorunlari yaratir.
```

Kural: Varsayılan olarak her zaman `const` kullan. Değeri değişecekse `let` kullan. `var` asla kullanma.

Arrow functions (ok fonksiyonları):

```
// Geleneksel fonksiyon
function add(a, b) {
  return a + b;
}

// Arrow function (kisa yazim)
const add = (a, b) => a + b;

// Cok satirli arrow function
const greet = (name) => {
  const message = `Merhaba, ${name}!`;
  return message;
};

// Tek parametre: parantez opsiyonel
const double = x => x * 2;
```

Template literals (şablon dizeler):

```
const name = "Ahmet";
const age = 22;

// Eski yontem (string birlistirme)
const msg1 = "Merhaba, " + name + "! Yasin: " + age;

// Template literal (backtick ile)
const msg2 = `Merhaba, ${name}! Yasin: ${age}`;
// Cok satirli dizeler de desteklenir
```

Destructuring (yapı bozma):

```
// Obje destructuring
const project = { title: "E-Ticaret", year: 2024 };
const { title, year } = project;
console.log(title); // "E-Ticaret"

```

```
// Dizi destructuring
const colors = ["red", "green", "blue"];
const [first, second] = colors;
console.log(first); // "red"

// Varsayilan deger
const { category = "Genel" } = project;

// Yeniden adlandirma
const { title: projectTitle } = project;
```

Spread ve Rest operatörleri (...):

```
// Spread: Diziyi / objeyi "acmak"
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]

const base = { color: "blue", size: "md" };
const extended = { ...base, size: "lg" };
// { color: "blue", size: "lg" }

// Rest: Geri kalanlari toplamak
const [head, ...tail] = [1, 2, 3, 4];
// head = 1, tail = [2, 3, 4]

function sum(...numbers) {
  return numbers.reduce((a, b) => a + b, 0);
}
```

Optional chaining (?.) ve Nullish coalescing (??):

```
const user = { profile: { name: "Ahmet" } };

// Optional chaining: null/undefined ise hata
// vermez, undefined dondurur
const city = user.profile?.address?.city;
// undefined (hata yok!)

// Nullish coalescing: null/undefined ise
// yedek deger kullan
const displayName = user.profile?.name ?? "Anonim";
// "Ahmet"

// || ile farki: || falsy degerleri de yakalar
// (0, "", false), ?? yalnizca null/undefined
const count = 0;
console.log(count || 10); // 10 (YANLIS!)
console.log(count ?? 10); // 0 (DOGRU)
```

Dikkat – Sık Yapılan Hatalar

- `===` kullan, `==` kullanma. `==` tip dönüşümü yapar ve beklenmedik sonuçlar verir: `"0" == false` → `true`.
- Arrow function'lar kendi `this` bağlamına sahip değildir. Class method'ları veya event handler'lar için bunu unutma.
- `??` ile `||` farklıdır. `0`, `""` ve `false` değerleri `||` ile kaybolur; `??` yalnızca `null`/`undefined` kontrol eder.

Sırada Ne Var?

Modern JavaScript sözdizimini hızlıca gözden geçirdik. Şimdi kodun modüler organizasyonunu sağlayan **ES Modules** konusuna geçelim.

3 ES Modules: import / export

Büyük bir projenin tüm kodunu tek bir dosyaya yazmak bakım kabusudur. **ES Modules** sayesinde kodunu mantıksal parçalara böler, her dosyadan yalnızca ihtiyacın olanı `import` edersin.

Teorik Arka Plan

Named export (isimli dışa aktarma):

```
// utils/format.ts
export const formatDate = (date: Date): string => {
  return date.toLocaleDateString("tr-TR");
};

export const formatCurrency = (amount: number) => {
  return `${amount.toLocaleString("tr-TR")} TL`;
};
```

```
// App.tsx --- Kullanim
import { formatDate, formatCurrency } from
  "./utils/format";

console.log(formatDate(new Date())); // "08.02.2025"
console.log(formatCurrency(1500));   // "1.500 TL"
```

Default export (varsayılan dışa aktarma):

```
// components/Card.tsx
export default function Card({ title, children }) {
  return (
    <div className="rounded-lg shadow p-4">
      <h3>{title}</h3>
      {children}
    </div>
  );
}
```

```
// App.tsx --- Kullanim (isim serbestce verilebilir)
import Card from "./components/Card";
import MyCard from "./components/Card"; // ayni sey
```

Karışık kullanım:

```
// services/api.ts
export const BASE_URL = "https://api.example.com";

export async function fetchProjects() { /* ... */ }

export default class ApiClient { /* ... */ }
```

```
// Hem default hem named import
import ApiClient, { BASE_URL, fetchProjects }
  from "./services/api";
```

Derinlemesine Anlama

Named export vs Default export: Hangisi ne zaman?

- **Named export:** Bir dosyadan birden fazla şey dışa aktarıyorsan. Utility fonksiyonlar, sabitler, tipler için ideal. Import ederken ismin eşleşmesi **zorunlu**; bu sayede otomatik tamamlama ve refactoring daha güvenli çalışır.
- **Default export:** Dosyanın "ana" çıktısı tek bir şeyse (bir React component, bir class). Import ederken istediğin ismi verebilirsin — ama bu bazen karışıklığa yol açar.
- **Pratik öneri:** React component dosyalarında default export, utility/servis dosyalarında named export kullan.

Dikkat – Sık Yapılan Hatalar

- Import yolları **göreceli** (relative) olmalıdır: `./utils/format`, `../components/Card`. Baştaki `./` veya `../` olmadan yazmak, node_modules'tan import anlamına gelir.
- TypeScript/React projesinde dosya uzantısını (`.ts`, `.tsx`) import'ta **yazmana gerek yok**; Vite otomatik çözer.
- Döngüsel import'lardan kaçın: A dosyası B'yi, B dosyası A'yı import ederse sorun çıkar. Modül yapını tek yönlü tutmaya çalış.

Sırada Ne Var?

Modül sistemini öğrendik. Şimdi web uygulamalarının en temel ihtiyacına geçiyoruz: **sunucudan veri çekmek**. Bunun için Fetch API ve async/await kullanacağız.

4 Async/Await ve Fetch API

Web uygulamaları veriyi genellikle bir sunucudan (API) çeker. Bu işlem ağ üzerinden yapıldığı için **asenkron** (eşzamansız) olmalıdır — veri gelene kadar sayfa donmamalı,

kullanıcı etkileşim edebilmelidir.

Teorik Arka Plan

Fetch API: Tarayıcının yerleşik HTTP istemcisidir. Bir URL'e istek gönderir ve `Promise` döner.
Promise: "İleride bir değer üreteceğini vaat eden" nesne. Üç durumu vardır:

- **pending:** İşlem devam ediyor.
- **fulfilled:** İşlem başarılı, değer hazır.
- **rejected:** İşlem başarısız, hata oluştu.

async/await: Promise'leri senkron görünümlü kodla yazmamızı sağlar.

```
// --- TEMEL FETCH ---
async function fetchProjects(): Promise<Project[]> {
  // 1. HTTP istegi gonder
  const response = await fetch("/data/projects.json");

  // 2. HTTP durumunu kontrol et
  if (!response.ok) {
    throw new Error(
      `HTTP Hata: ${response.status}`
    );
  }

  // 3. JSON'a donustur
  const data: Project[] = await response.json();

  return data;
}
```

Adım adım ne oluyor:

1. `fetch()` bir HTTP GET isteği gönderir ve `Response` nesnesi döner.
2. `await` anahtar kelimesi, Promise çözülene kadar bekler (ama ana thread'i **bloklamaz**).
3. `response.ok`, HTTP durum kodunun 200–299 aralığında olduğunu kontrol eder.
4. `response.json()`, yanıt gövdesini JSON olarak ayrıştırır (parse eder) ve bir Promise döner.
5. Fonksiyonun başındaki `async` anahtar kelimesi, içeride `await` kullanılacağını bildirir.

Benzetme ile Anlama

Bir restoranda sipariş verirsin. Garson mutfağa gider (fetch isteği), yemek hazırlanır (pending), garson tabağı getirir (fulfilled) veya "malzeme bitti" der (rejected). `await`, "garson gelene kadar masada otur ama sohbet etmeye devam et" demektir — sayfa donmaz.

Uygulama Adımı

Uygulama-1: Yerel JSON Dosyası Oluştur ve Fetch Et

1. Proje klasöründe `public/data/projects.json` dosyası oluştur:

```
[
  {
    "id": 1,
    "title": "E-Ticaret Platformu",
    "description": "React ve Node.js ile
      tam kapsamli e-ticaret uygulamasi.",
    "tech": ["React", "Node.js", "MongoDB"],
    "year": 2024,
    "category": "fullstack",
    "featured": true,
    "image": "/images/ecommerce.jpg"
  },
  {
    "id": 2,
    "title": "Blog Uygulamasi",
    "description": "Next.js ile SEO dostu
      kisisel blog platformu.",
    "tech": ["Next.js", "TypeScript", "Tailwind"],
    "year": 2024,
    "category": "frontend",
    "featured": false,
    "image": "/images/blog.jpg"
  },
  {
    "id": 3,
    "title": "Hava Durumu API",
    "description": "OpenWeather API ile
      anlik hava durumu sorgulama.",
    "tech": ["JavaScript", "REST API", "CSS"],
    "year": 2023,
    "category": "frontend",
    "featured": true,
    "image": "/images/weather.jpg"
  },
  {
    "id": 4,
    "title": "Gorev Yoneticisi",
    "description": "Drag-and-drop destekli
      Kanban tarzi gorev panosu.",
    "tech": ["React", "TypeScript", "DnD Kit"],
    "year": 2024,
    "category": "frontend",
    "featured": false,
    "image": "/images/taskboard.jpg"
  },
  {
    "id": 5,
    "title": "Chat Uygulamasi",
    "description": "WebSocket ile gercek zamanli
      mesajlasma.",
```

```
    "tech": ["React", "Socket.io", "Express"],
    "year": 2023,
    "category": "fullstack",
    "featured": true,
    "image": "/images/chat.jpg"
  }
]
```

2. Bu JSON dosyasını React component'inde fetch et (detaylı kod Bölüm 11'de).

Sırada Ne Var?

Fetch API ile veri çekmeyi öğrendik. Peki ya istek başarısız olursa? Ağ kesilirse? JSON bozuksa? Bir sonraki bölümde **hata yönetimi** konusunu ele alacağız.

5 Error Handling (Hata Yönetimi)

Gerçek dünyada her şey her zaman başarılı olmaz: Ağ bağlantısı kesilebilir, API yanıt vermeyebilir, JSON formatı bozuk olabilir. İyi bir geliştirici bu senaryolara **hazırlıklıdır**.

Teorik Arka Plan

try / catch / finally:

```javascript
async function loadProjects() {
  try {
    // Hata olusabilecek kod
    const response = await fetch(
      "/data/projects.json"
    );

    if (!response.ok) {
      throw new Error(
        `Sunucu hatasi: ${response.status}`
      );
    }

    const projects = await response.json();
    return projects;

  } catch (error) {
    // Hata yakalanir ve islenir
    if (error instanceof TypeError) {
      // Ag hatasi (fetch basarisiz)
      console.error("Ag baglantisi yok:", error);
    } else {
      console.error("Bilinmeyen hata:", error);
    }
    return []; // Bos dizi don (fallback)

```

```
  } finally {
    // Her durumda calisir (basarili veya hatali)
    console.log("Yukleme islemi tamamlandi.");
  }
}
```

Hata türleri:

- **TypeError:** Ağ bağlantısı hatası (`fetch` isteği hiç gönderilemedi).
- **SyntaxError:** JSON ayrıştırma hatası (`response.json()` başarısız).
- **Özel Error:** Biz atıyoruz (`throw new Error(...)`).
- **HTTP Hatası:** `response.ok` false (404, 500 vb.) — `fetch` bunu hata **olarak atmaz**, manuel kontrol gerekir!

Derinlemesine Anlama

Önemli: `fetch` yalnızca ağ hatasında reject olur!
`fetch("/api/users")` isteğinde sunucu 404 veya 500 döndürse bile Promise **resolve** olur (çünkü ağ iletişimi başarılı). Bu yüzden `response.ok` kontrolü **zorunludur**. Bunu yapmayan geliştiriciler, hatalı veriyi "başarılı" sanır ve bug'lara yol açar.

```
// YANLIS: Hata kontrolu yok
const data = await (await fetch(url)).json();

// DOGRU: Hata kontrolu var
const response = await fetch(url);
if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}
const data = await response.json();
```

Dikkat – Sık Yapılan Hatalar

- `catch` bloğunda hatayı **sessizce yutma** (`catch {}`). En azından `console.error` ile logla; tercihen kullanıcıya hata mesajı göster.
- `finally` bloğu, loading spinner'ını kapatmak için idealdir — hem başarılı hem başarısız durumda çalışır.
- Async fonksiyondan hata fırlatırsan, çağıran fonksiyonun da `try/catch` ile sarması gerekir — hata zinciri boyunca yukarı akar.

Sırada Ne Var?

JavaScript temellerini tamamladık. Şimdi büyük sıçramayı yapıyoruz: **TypeScript**. JavaScript'in üstüne **tip güvenliği** ekleyerek hataları daha kod çalışmadan yakalayacağız.

6 TypeScript'e Giriş: Neden Tip Güvenliği?

JavaScript **dinamik tipli** bir dildir: Bir değişkene önce sayı, sonra string, sonra dizi atayabilirsin — dil şikayet etmez. Bu esneklik küçük projelerde hız kazandırır ama büyük projelerde **bug üretir**.

Teorik Arka Plan

Problem: JavaScript'te tip hatası

```
// JavaScript --- hata derleme zamaninda yok,
// calisma zamaninda (runtime) patlak!
function getFullName(user) {
  return user.firstName + " " + user.lastName;
}

// Yanlis kullanim: hatayi ancak
// calistirinca gorursun
getFullName({ name: "Ahmet" });
// "undefined undefined" --- bug!
getFullName(null);
// TypeError: Cannot read properties of null
```

Çözüm: TypeScript'te tip güvenliği

```
// TypeScript --- hata YAZARKEN gorulur
interface User {
  firstName: string;
  lastName: string;
}

function getFullName(user: User): string {
  return user.firstName + " " + user.lastName;
}

getFullName({ name: "Ahmet" });
// DERLEME HATASI: 'name' property does not
// exist on type 'User'. Did you mean
// 'firstName'?

getFullName(null);
// DERLEME HATASI: Argument of type 'null' is
// not assignable to parameter of type 'User'.
```

Hata, kod **çalışmadan** — hatta kaydettiğin anda — editörde kırmızı alt çizgi ile görülür.

Gerçek Hayatta Karşılığı

TypeScript, kodun "yazım kılavuzu" gibidir. Bir mimari projesinde "bu boru 2 inç" yazmak yerine sadece "boru" yazsaydınız, tesisatçı hangi boruyu kullanacağını bilemezdi. TypeScript, her değerin "boyutunu" (tipini) açıkça belirtir — böylece yanlış

parça takılmaz.

Derinlemesine Anlama

TypeScript nedir, tam olarak?

- TypeScript, JavaScript'in **üst kümesidir** (superset). Her geçerli JavaScript kodu aynı zamanda geçerli TypeScript kodudur.
- TypeScript **derleme zamanında** (compile-time) çalışır; tarayıcıya gitmeden önce JavaScript'e dönüştürülür.
- Tarayıcılar TypeScript'i doğrudan çalıştır**amaz**. Vite (veya tsc) TypeScript'i JavaScript'e derler.
- Tip kontrolleri yalnızca **geliştirme zamanında** çalışır; üretim (production) kodunda tipler **silinir** ve sıfır performans maliyeti olur.

Sırada Ne Var?

TypeScript'in "neden" kısmını anladık. Şimdi "nasıl" kısmına geçelim: Temel tipler ve type annotations.

7 TypeScript Temel Tipler

TypeScript'te her değişken, parametre ve dönüş değeri bir **tip** ile etiketlenebilir. Bu etiketlemeye **type annotation** denir.

Teorik Arka Plan

Primitif tipler:

```
// Temel tipler
const name: string = "Ahmet";
const age: number = 22;
const isStudent: boolean = true;

// Diziler
const scores: number[] = [85, 92, 78];
const names: string[] = ["Ali", "Veli"];
// Alternatif: Array<number>

// Tuple (sabit uzunluklu, farkli tipli dizi)
const pair: [string, number] = ["Ahmet", 22];

// null ve undefined
const emptyValue: null = null;
const notDefined: undefined = undefined;
```

Union tipler (birleşim):

```
// Birden fazla tip kabul eden degisken
let status: "loading" | "success" | "error";
status = "loading"; // gecerli
status = "success"; // gecerli
```

```
status = "pending"; // HATA! Bu deger izin verilmez

// ıSay veya null olabilir
let count: number | null = null;
count = 42; // gecerli
count = "abc"; // HATA!
```

Literal tipler:

```
// Sadece belirli degerlere izin ver
type Category = "frontend" | "fullstack" | "backend";
type Size = "sm" | "md" | "lg";

const projectCategory: Category = "frontend"; // OK
const projectCategory2: Category = "mobile";
// HATA: '"mobile"' is not assignable to
// type 'Category'
```

Fonksiyonlarda tip tanımlama:

```
// Parametre ve donus tipi
function add(a: number, b: number): number {
  return a + b;
}

// Arrow function
const multiply = (a: number, b: number): number =>
  a * b;

// Opsiyonel parametre (? ile)
function greet(name: string, title?: string): string {
  if (title) {
    return `${title} ${name}`;
  }
  return `Merhaba, ${name}`;
}

greet("Ahmet");            // "Merhaba, Ahmet"
greet("Ahmet", "Dr.");     // "Dr. Ahmet"

// Varsayilan deger
function paginate(
  items: unknown[],
  page: number = 1,
  limit: number = 10
) {
  // ...
}
```

Dikkat – Sık Yapılan Hatalar

- `any` tipinden kaçın. `any` TypeScript'in tip kontrolünü tamamen devre dışı bırakır — JavaScript yazmaktan farkı kalmaz. Gerçekten tipi bilmiyorsan `unknown` kullan ve `typeof` kontrolü yap.
- TypeScript **tip çıkarımı** (type inference) yapar: `const x = 42` yazdığında TypeScript otomatik olarak `x: number` bilir. Her yere tip yazmak **gerekmez**; ama fonksiyon parametreleri için **her zaman** yaz.
- `string` (küçük harf) kullan, `String` (büyük harf) kullanma. `String` wrapper nesnedir, `string` primitif tiptir.

Sırada Ne Var?

Temel tipleri öğrendik. Şimdi karmaşık veri yapılarını tanımlamamızı sağlayan **interface** ve **type alias** kavramlarına geçelim.

8 Interface ve Type Alias

Tek bir `string` veya `number` yetmez; gerçek uygulamalarda veriler çok alanlı **objeler** halindedir. TypeScript'te bu objelerin şeklini (shape) tanımlamak için `interface` veya `type` kullanırız.

Teorik Arka Plan

Interface:

```
interface Project {
  id: number;
  title: string;
  description: string;
  tech: string[];
  year: number;
  category: "frontend" | "fullstack" | "backend";
  featured: boolean;
  image: string;
}

// Kullanim
const project: Project = {
  id: 1,
  title: "E-Ticaret",
  description: "Tam kapsamli uygulama",
  tech: ["React", "Node.js"],
  year: 2024,
  category: "fullstack",
  featured: true,
  image: "/images/ecommerce.jpg",
};

// HATA: 'category' degeri yanlis
```

```
const bad: Project = {
  // ...
  category: "mobile", // Type '"mobile"' is not
  // assignable to type '"frontend" | ...'
};
```

Opsiyonel alanlar (?):

```
interface Project {
  id: number;
  title: string;
  description: string;
  tech: string[];
  year: number;
  category: "frontend" | "fullstack" | "backend";
  featured: boolean;
  image: string;
  // Opsiyonel alanlar
  demoUrl?: string;    // olabilir de olmayabilir
  sourceUrl?: string;  // olabilir de olmayabilir
}
```

Readonly alanlar:

```
interface Project {
  readonly id: number; // Olusturulduktan sonra
                       // degistirilemez
  title: string;
  // ...
}

const p: Project = { id: 1, title: "Test" };
p.id = 2; // HATA: Cannot assign to 'id'
          // because it is a read-only property
```

Type alias:

```
// type alias ile ayni sey
type Category = "frontend" | "fullstack" | "backend";

type Project = {
  id: number;
  title: string;
  category: Category;
  tech: string[];
  // ...
};

// Union ve intersection type'lar
// yalnizca type ile yapilabilir
type ApiResponse<T> =
  | { status: "success"; data: T }
```

```
  | { status: "error"; message: string };

// Intersection: iki tipi birlestir
type WithTimestamp = Project & {
  createdAt: Date;
  updatedAt: Date;
};
```

Derinlemesine Anlama

Interface vs Type: Hangisi ne zaman?

- **Interface:** Obje şekillerini tanımlamak için. `extends` ile genişletilebilir. React component prop'ları için geleneksel tercih.
- **Type:** Union, intersection, mapped type gibi gelişmiş tip operasyonları için. Primitif tip alias'ları için (`type ID = string | number`).
- **Pratik kural:** Obje tanımlıyorsan `interface`, karmaşık tip operasyonu yapıyorsan `type` kullan. İkisi çoğu durumda birbirinin yerine geçer.

Uygulama Adımı

Uygulama-2: Project Veri Modelini Tanımla

`src/types/project.ts` dosyası oluştur:

```
// Kategori tipleri
export type Category =
  | "frontend"
  | "fullstack"
  | "backend";

// Siralama secenekleri
export type SortField = "year" | "title";
export type SortOrder = "asc" | "desc";

// Proje veri modeli
export interface Project {
  readonly id: number;
  title: string;
  description: string;
  tech: string[];
  year: number;
  category: Category;
  featured: boolean;
  image: string;
  demoUrl?: string;
  sourceUrl?: string;
}

// Filtre durumu
export interface FilterState {
```

```
  search: string;
  category: Category | "all";
  sortField: SortField;
  sortOrder: SortOrder;
}
```

Sırada Ne Var?

Interface ve type alias ile veri modelimizi tanımladık. Şimdi TypeScript'in en güçlü özelliklerinden birine kısaca göz atalım: **Generics**.

9 Generics (Temel Düzey)

Bazen bir fonksiyon birden fazla tipte veri ile çalışmalıdır ama tip güvenliğini kaybetmemelidir. **Generics**, "tipi parametre olarak alan" yapılardır.

Teorik Arka Plan

Problem: Tip güvenliği vs esneklik

```
// SORUN 1: Her tip icin ayri fonksiyon yazmak
function getFirstNumber(arr: number[]): number {
  return arr[0];
}
function getFirstString(arr: string[]): string {
  return arr[0];
}

// SORUN 2: any kullanmak (tip guvenligi yok)
function getFirst(arr: any[]): any {
  return arr[0]; // donus tipi bilinmiyor!
}
```

Çözüm: Generic fonksiyon

```
// T bir "tip parametresi" --- cagrildiginda
// gercek tipe donusur
function getFirst<T>(arr: T[]): T {
  return arr[0];
}

// Kullanim: TypeScript T'yi otomatik cikarir
const num = getFirst([1, 2, 3]);
// num: number (T = number)

const str = getFirst(["a", "b", "c"]);
// str: string (T = string)

const project = getFirst(projects);
// project: Project (T = Project)
```

Generic interface:

```
// API yaniti icin generic tip
interface ApiResponse<T> {
  data: T;
  total: number;
  page: number;
}

// Kullanim
type ProjectsResponse =
  ApiResponse<Project[]>;
// { data: Project[], total: number, page: number }

type UserResponse = ApiResponse<User>;
// { data: User, total: number, page: number }
```

Generic ile kısıtlama (extends):

```
// T en az 'id' alanina sahip olmali
function findById<T extends { id: number }>(
  items: T[],
  id: number
): T | undefined {
  return items.find(item => item.id === id);
}

// Calisiir: Project'te id var
const project = findById(projects, 3);

// HATA: string[] 'id' alanina sahip degil
const str = findById(["a", "b"], 1);
```

Benzetme ile Anlama

Generic'leri bir **kalıp** gibi düşün. Kurabiye kalıbı yıldız şeklinde — ama hamuru sen seçersin: çikolatalı, sade, baharatlı. Kalıp (generic fonksiyon) aynı, malzeme (tip) değişir. Sonuç her zaman yıldız şeklinde (tip-güvenli) olur ama içerik farklıdır.

Dikkat – Sık Yapılan Hatalar

- Generic parametre isimleri geleneksel olarak tek harftir: `T` (Type), `K` (Key), `V` (Value), `E` (Element). Ama anlamlı isimler de kullanabilirsin: `TData`, `TResponse`.
- Generics'i her yerde kullanma — basit fonksiyonlarda gereksiz karmaşıklık yaratır. Yalnızca birden fazla tipte çalışması gereken **yeniden kullanılabilir** yapılarda kullan.
- JSX dosyalarında (`.tsx`) arrow function generic'i `<T,>` şeklinde yazılır (virgül ile): `const fn = <T,>(x: T) => x;`. Virgül olmadan JSX tag'i sanılır.

Sırada Ne Var?

Generics ile tip-güvenli, yeniden kullanılabilir yapılar oluşturmayı öğrendik. Şimdi "DOM yerine state" yaklaşımını ve React'te veri yönetimini anlayalım.

10 DOM Yerine "State" Mantığı

Geleneksel JavaScript'te UI güncellemek için doğrudan DOM manipülasyonu yapılır: `document.getElementById`, `innerHTML`, `appendChild`... Bu yaklaşım küçük projelerde çalışır ama büyük projelerde kaosa yol açar.

Teorik Arka Plan

DOM manipülasyonu (eski yol):

```
// HER degisiklikte DOM'u elle guncelle
function renderProjects(projects) {
  const container =
    document.getElementById("project-list");
  container.innerHTML = ""; // Temizle

  projects.forEach(project => {
    const card = document.createElement("div");
    card.className = "project-card";
    card.innerHTML = `
      <h3>${project.title}</h3>
      <p>${project.description}</p>
    `;
    container.appendChild(card);
  });
}

// Filtreleme
document.getElementById("search")
  .addEventListener("input", (e) => {
    const filtered = projects.filter(p =>
      p.title.includes(e.target.value)
    );
    renderProjects(filtered); // YENiDEN ciz!
  });
```

Sorunlar:

- Her değişiklikte tüm listeyi silip yeniden çiziyorsun — verimsiz.
- Veri (projects) ve görünüm (DOM) birbirine karışıyor — debug zor.
- 10 farklı etkileşim varsa 10 ayrı event listener ve 10 ayrı render fonksiyonu yazman gerekiyor.

State yaklaşımı (React yolu):

```
function ProjectList() {
  // DURUM (state) tanimi
  const [projects, setProjects] =
```

```
    useState<Project[]>([]);
  const [search, setSearch] = useState("");
  const [category, setCategory] =
    useState<Category | "all">("all");

  // Filtrelenmis veri (state'ten TURETILIR)
  const filtered = projects
    .filter(p =>
      p.title.toLowerCase()
        .includes(search.toLowerCase())
    )
    .filter(p =>
      category === "all"
        ? true
        : p.category === category
    );

  // UI, state'in bir FONKSIYONUDUR
  // State degistiginde React otomatik
  // gunceller --- sen DOM'a dokunmuyorsun
  return (
    <div>
      <input
        value={search}
        onChange={e => setSearch(e.target.value)}
        placeholder="Proje ara..."
      />
      {filtered.map(p => (
        <Card key={p.id} title={p.title}>
          {p.description}
        </Card>
      ))}
    </div>
  );
}
```

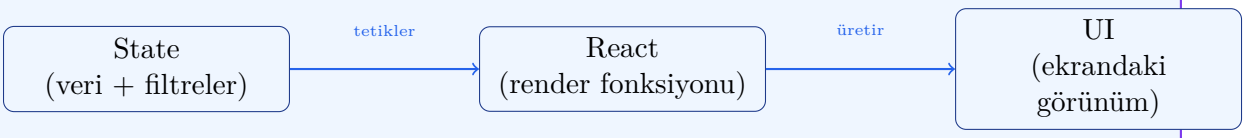
Fark:

- **Veri (state)** ve **görünüm (JSX)** ayrı. Sen sadece state'i güncellersin; React DOM'u otomatik günceller.
- Filtreleme, sıralama gibi işlemler state'ten **türetilir** (derived state) — ayrı render fonksiyonu yazmana gerek yok.
- React yalnızca **değişen** kısmı günceller (Virtual DOM diff) — tüm listeyi silip yeniden çizmez.

Derinlemesine Anlama

"UI = f(state)" Formülü

React'in temel felsefesi şudur: Kullanıcı arayüzü (UI), durumun (state) bir **fonksiyonudur**. Aynı state verildiğinde her zaman aynı UI çıkar. Bu, UI'ı **öngörülebilir** ve **test edilebilir** kılar.



State değiştiğinde → React yeniden render eder → UI güncellenir. Sen DOM'a **asla** dokunmazsın.

Sırada Ne Var?

State mantığını kavradık. Şimdi tüm bilgileri birleştirerek ana uygulamayı yazacağız: JSON'dan proje verisi çekip, TypeScript tipleri ile modelleyip, filtreleme ve sıralama yapacağız.

11 Uygulama: JSON'dan Proje Verisi Çekme, Filtreleme ve Sıralama

Bu bölümde tüm LAB-5 bilgilerini birleştirerek tam çalışan bir proje listeleme sayfası oluşturacaksın.

Uygulama Adımı

Uygulama-3: Servis Katmanı (API İstekleri)
`src/services/projectService.ts` dosyası oluştur:

```
import type { Project } from "../types/project";

const API_URL = "/data/projects.json";

export async function fetchProjects():
  Promise<Project[]> {
  try {
    const response = await fetch(API_URL);

    if (!response.ok) {
      throw new Error(
        `Projeler yuklenemedi: ${response.status}`
      );
    }

    const data: Project[] =
      await response.json();
    return data;

  } catch (error) {
    console.error("Veri cekme hatasi:", error);
    throw error; // Hatayi yukari ilet
  }
}
```

Uygulama Adımı

Uygulama-4: Yardımcı Fonksiyonlar (Filtreleme ve Sıralama)
`src/utils/projectHelpers.ts` dosyası oluştur:

```
import type {
  Project, Category, SortField, SortOrder
} from "../types/project";

// --- Arama filtresi ---
export function filterBySearch(
  projects: Project[],
  query: string
): Project[] {
  if (!query.trim()) return projects;

  const lower = query.toLowerCase();
  return projects.filter(p =>
    p.title.toLowerCase().includes(lower) ||
    p.description.toLowerCase().includes(lower) ||
    p.tech.some(t =>
      t.toLowerCase().includes(lower)
    )
  );
}

// --- Kategori filtresi ---
export function filterByCategory(
  projects: Project[],
  category: Category | "all"
): Project[] {
  if (category === "all") return projects;
  return projects.filter(
    p => p.category === category
  );
}

// --- Siralama ---
export function sortProjects(
  projects: Project[],
  field: SortField,
  order: SortOrder
): Project[] {
  const sorted = [...projects].sort((a, b) => {
    if (field === "year") {
      return a.year - b.year;
    }
    return a.title.localeCompare(b.title, "tr");
  });

  return order === "desc"
```

```
    ? sorted.reverse()
    : sorted;
}

// --- Hepsini birlestir ---
export function applyFilters(
  projects: Project[],
  search: string,
  category: Category | "all",
  sortField: SortField,
  sortOrder: SortOrder
): Project[] {
  let result = filterBySearch(projects, search);
  result = filterByCategory(result, category);
  result = sortProjects(result, sortField,
    sortOrder);
  return result;
}
```

Uygulama Adımı

Uygulama-5: React Component (Ana Sayfa)
`src/App.tsx` dosyasını güncelle:

```
import { useState, useEffect } from "react";
import type {
  Project, Category, SortField, SortOrder
} from "./types/project";
import { fetchProjects } from
  "./services/projectService";
import { applyFilters } from
  "./utils/projectHelpers";
import Card from "./components/Card";
import Input from "./components/Input";
import Button from "./components/Button";
import Alert from "./components/Alert";

export default function App() {
  // --- STATE ---
  const [projects, setProjects] =
    useState<Project[]>([]);
  const [search, setSearch] = useState("");
  const [category, setCategory] =
    useState<Category | "all">("all");
  const [sortField, setSortField] =
    useState<SortField>("year");
  const [sortOrder, setSortOrder] =
    useState<SortOrder>("desc");
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] =
    useState<string | null>(null);

  // --- VERi CEKME ---
  useEffect(() => {
    async function load() {
      try {
        setLoading(true);
        setError(null);
        const data = await fetchProjects();
        setProjects(data);
      } catch (err) {
        setError(
          err instanceof Error
            ? err.message
            : "Bilinmeyen hata"
        );
      } finally {
        setLoading(false);
      }
    }
    load();
  }, []);

  // --- TURETILMIS (DERIVED) VERi ---
  const filtered = applyFilters(
    projects, search, category,
    sortField, sortOrder
  );

  const categories: (Category | "all")[] =
    ["all", "frontend", "fullstack", "backend"];

  // --- UI ---
  return (
    <div className="min-h-screen bg-gray-50
      dark:bg-gray-950 p-4 md:p-8">
      <div className="max-w-6xl mx-auto">
        <h1 className="text-3xl font-bold
          text-gray-900 dark:text-white mb-8">
          Projelerim
        </h1>

        {/* HATA DURUMU */}
        {error && (
          <Alert variant="error" title="Hata">
            {error}
          </Alert>
        )}

```

```
        {/* FILTRELER */}
        <div className="flex flex-col sm:flex-row
          gap-4 mb-8">
          <Input id="search"
            placeholder="Proje ara..."
            value={search}
            onChange={e =>
              setSearch(e.target.value)} />

          <div className="flex gap-2 flex-wrap">
            {categories.map(cat => (
              <Button key={cat}
                variant={category === cat
                  ? "primary" : "ghost"}
                size="sm"
                onClick={() => setCategory(cat)}>
                {cat === "all"
                  ? "Tumu" : cat}
              </Button>
            ))}
          </div>

          <div className="flex gap-2">
            <select value={sortField}
              onChange={e => setSortField(
                e.target.value as SortField
              )}
              className="border rounded-lg px-3
                py-2 dark:bg-gray-800
                dark:text-white">
              <option value="year">Yil</option>
              <option value="title">Baslik</option>
            </select>
            <Button variant="ghost" size="sm"
              onClick={() => setSortOrder(
                o => o === "asc" ? "desc" : "asc"
              )}>
              {sortOrder === "asc"
                ? "A-Z" : "Z-A"}
            </Button>
          </div>
        </div>

        {/* YUKLENIYOR */}
        {loading && (
          <p className="text-center
            text-gray-500">
            Yukleniyor...
          </p>
        )}
```

```
        {/* PROJE LISTESI */}
        {!loading && filtered.length === 0 && (
          <p className="text-center
            text-gray-500">
            Eslesen proje bulunamadi.
          </p>
        )}

        <div className="grid grid-cols-1
          sm:grid-cols-2 lg:grid-cols-3 gap-6">
          {filtered.map(project => (
            <Card key={project.id}
              variant="elevated"
              title={project.title}
              image={project.image}
              imageAlt={`${project.title}
                ekran goruntusu`}>
              <p className="text-sm mb-3">
                {project.description}
              </p>
              <div className="flex flex-wrap
                gap-1">
                {project.tech.map(t => (
                  <span key={t}
                    className="bg-blue-100
                      dark:bg-blue-900
                      text-blue-800
                      dark:text-blue-200
                      text-xs px-2 py-0.5
                      rounded-full">
                    {t}
                  </span>
                ))}
              </div>
              <p className="text-xs
                text-gray-400 mt-2">
                {project.year} &middot;
                {project.category}
              </p>
            </Card>
          ))}
        </div>

        {/* SONUC SAYISI */}
        <p className="text-sm text-gray-500
          mt-4 text-center">
          {filtered.length} / {projects.length}
          {" "}proje gosteriliyor
        </p>
```

```
      </div>
    </div>
  );
}
```

Dikkat – Sık Yapılan Hatalar

- `useEffect` içinde doğrudan `async` callback yazamazsın. İç içe bir async fonksiyon tanımlayıp çağırman gerekir (yukarıdaki `load()` fonksiyonu gibi).
- `key` prop'u her `.map()` içinde **zorunludur**. `key={project.id}` kullan; `key={index}` kullanma (sıralama değiştiğinde sorun çıkar).
- `as SortField` gibi type assertion'ları dikkatli kullan. Yanlış assertion TypeScript'in korumalarını devre dışı bırakır.

Sırada Ne Var?

Tam çalışan bir proje listeleme sayfası oluşturduk: JSON'dan veri çekiyor, TypeScript tipleri ile güvence altında, filtreleme ve sıralama state ile yönetiliyor. Şimdi Git ile teslim edeceğiz.

12 Git İş Akışı ve Teslim

Uygulama Adımı

Uygulama-6: Git ile Kaydet ve GitHub'a Yükle

1. Yeni branch aç:

```
git checkout -b feature/typescript-projects
```

2. Adım adım commit at:

```
git add src/types/
git commit -m "feat: add Project TypeScript types"

git add public/data/projects.json
git commit -m "feat: add project mock data JSON"

git add src/services/
git commit -m "feat: add project fetch service"

git add src/utils/projectHelpers.ts
git commit -m "feat: add filter and sort helpers"

git add src/App.tsx
git commit -m "feat: implement project listing
  with filter/sort"
```

3. GitHub'a push et:

```
git push -u origin feature/typescript-projects
```

13 Teslim Kontrol Listesi ve Değerlendirme Rubriği

Kontrol Listesi

Teslim öncesi kontrol listesi:

- ☐ TypeScript tipleri tanımlanmış mı? (`Project`, `Category`, `FilterState` vb.)
- ☐ `any` tipi **hiçbir yerde** kullanılmamış mı?
- ☐ JSON veri dosyası (`public/data/projects.json`) en az 5 proje içeriyor mu?
- ☐ Fetch API ile veri çekme çalışıyor mu?
- ☐ Hata durumu (error state) kullanıcıya gösterilmiş mi?
- ☐ Yükleniyor (loading) durumu gösterilmiş mi?
- ☐ Arama (search) filtresi çalışıyor mu?
- ☐ Kategori filtresi çalışıyor mu?
- ☐ Sıralama (sort) çalışıyor mu?
- ☐ Kod modüler mi? (types, services, utils, components ayrı dosyalarda)
- ☐ TypeScript derlemede hata yok mu? (`npx tsc --noEmit`)
- ☐ En az 3 anlamlı commit atılmış ve branch kullanılmış mı?
- ☐ GitHub'a push edilmiş mi?

13.1 Değerlendirme Rubriği

Kriter	Puan	Açıklama
TypeScript tipleri doğru tanımlı	15	interface/type, union, literal
Fetch API ile veri çekme	10	async/await, response.ok kontrolü
Error handling (try/catch)	10	Hata mesajı kullanıcıya gösterilir
Loading state yönetimi	5	Yükleniyor göstergesi
Arama filtresi	15	Başlık, açıklama, teknolojide arama
Kategori filtresi	10	all / frontend / fullstack / backend
Sıralama	10	Yıla ve başlığa göre, asc/desc
Modüler kod yapısı	10	types, services, utils ayrı
`any` kullanılmamış	5	Sıfır any tipi
Git iş akışı (branch + commit)	10	En az 3 anlamlı commit
Toplam	**100**	

14 Kendini Test Et

Kendini Test Et

Aşağıdaki sorulara kendi cümlelerinle 2–3 satır cevap yazabiliyor musun? Yazamıyorsan ilgili bölümü tekrar oku.

1. **`const` ile `let` farkı nedir? `var` neden kullanılmamalıdır?**
2. **Optional chaining (`?.`) ve nullish coalescing (`??`) ne işe yarar? `||` ile `??` farkı nedir?**
3. **Named export ile default export arasındaki fark nedir? Hangisi ne zaman tercih edilir?**
4. **`async/await` ne sağlar? `await` anahtar kelimesi olmadan `fetch` çağırırsan ne olur?**
5. **`fetch()` neden 404 hatasında reject olmaz? `response.ok` kontrolü neden zorunludur?**
6. **TypeScript `any` tipi neden tehlikelidir? Alternatif olarak ne kullanılmalıdır?**
7. **Interface ile type alias arasındaki fark nedir? Obje tanımlarken hangisi tercih edilir?**
8. **Generic (`<T>`) nedir? Neden `any` yerine generic kullanmalıyız?**
9. **"UI = f(state)" formülü ne anlama gelir? DOM manipülasyonu yerine state kullanmanın avantajı nedir?**
10. **`useEffect` içinde neden doğrudan async fonksiyon yazılamaz?**

15 Ek Kaynaklar ve İleri Okuma

- **TypeScript Handbook:** https://www.typescriptlang.org/docs/handbook/ — Resmi ve kapsamlı TypeScript rehberi.
- **TypeScript Playground:** https://www.typescriptlang.org/play — Tarayıcıda TypeScript denemeleri.
- **JavaScript.info:** https://javascript.info — Modern JavaScript'in en kapsamlı ücretsiz rehberi.
- **MDN Fetch API:** https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch — Fetch API detaylı dokümantasyonu.
- **MDN Async/Await:** https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous/Promises Promise ve async/await rehberi.
- **Total TypeScript (Beginners):** https://www.totaltypescript.com/tutorials — Matt Pocock'un ücretsiz TypeScript eğitim serisi.
- **React + TypeScript Cheatsheet:** https://react-typescript-cheatsheet.netlify.app — React'te TypeScript kullanımı için pratik rehber.
- **Type Challenges:** https://github.com/type-challenges/type-challenges — TypeScript tip sistemi alıştırmaları (ileri seviye).